

Wedding Photo Sharing

MVP Architecture & Development Roadmap - v2

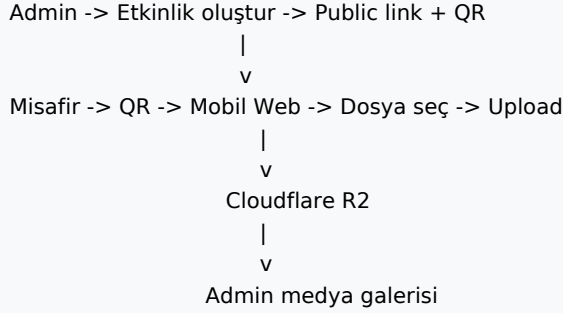
Codex'in projeyi tek seferde anlayıp, aktif milestone'a göre hızlı ve kontrollü biçimde geliştirebilmesi için ana mimari ve uygulama belgesi.

Ürün	QR ile etkinlik fotoğraf/video toplama
Ana hedef	Hızlı çalışan ticari MVP
Kullanım	Misafir login olmadan mobil web'den upload
Öncelik	iPhone Safari / Android Chrome
Backend	Java 21 + Spring Boot
Frontend	React + Vite
Database	PostgreSQL
Storage	Cloudflare R2 - S3 compatible
Deployment	Docker Compose + tek VPS
CI/CD	GitHub -> Jenkins -> Registry -> VPS
Mimari	Modular monolith / MVP-first

1. Ürün Vizyonu

Wedding Photo Sharing; düğün, nişan ve benzeri etkinliklerde misafirlerin telefonlarıyla çektikleri fotoğraf ve videoları WhatsApp veya Instagram üzerinden tek tek göndermesine gerek bırakmadan, QR kod ile merkezi şekilde toplamayı amaçlayan web tabanlı bir üründür.

Misafir deneyimi: QR okut -> etkinlik sayfası aç -> fotoğraf/video seç -> yükle -> tamamlandı mesajı.
Misafir hesabı, Google hesabı veya uygulama kurulumu yoktur.



MVP başarı kriterleri

- Admin etkinlik oluşturabilmeli ve benzersiz public link/QR alabilmeli.
- Misafir iPhone Safari ve Android Chrome'dan birden fazla fotoğraf/video yükleyebilmeli.
- Upload sırasında dosya bazlı progress, hata ve retry görülebilmeli.
- Admin yüklenen medyayı tekil, seçili grup veya tüm etkinlik olarak indirebilmeli.
- Sistem localde Docker Compose ile ayağa kalkmalı.
- main branch push sonrası Jenkins build/test/image/deploy hattı production VPS'i güncelleyebilmeli.

2. MVP Kapsamı

MVP'de olacak

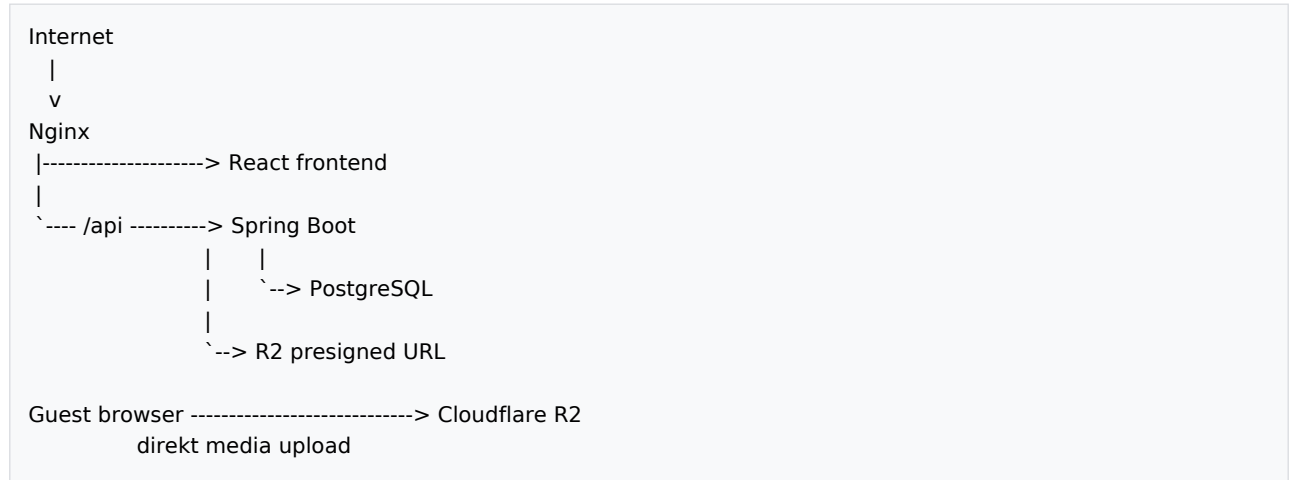
- Basit admin authentication.
- Etkinlik oluşturma, listeleme, görüntüleme ve temel güncelleme.
- Etkinlik adı, tarih, kapak fotoğrafı ve aktif/pasif durumu.
- Benzersiz public token ve QR kod.
- Login gerektirmeyen mobil misafir sayfası.
- Çoklu fotoğraf/video seçimi ve doğrudan R2 upload.
- Admin medya galerisi.
- Tekil indirme.
- Checkbox ile seçilen medyaları ZIP olarak indirme.
- Etkinlikteki tüm medyayı ZIP olarak indirme.
- PostgreSQL persistence.
- Docker Compose local ve production çalışma.
- Basit Jenkins CI/CD: test, build, Docker image, registry push, VPS deploy.

MVP'de olmayacak

- Microservice mimarisi.
- Kubernetes, Kafka, RabbitMQ veya Redis.
- Blue/green, canary veya multi-stage enterprise deployment.
- ArgoCD, Terraform veya karmaşık approval workflow.
- Ödeme/paket sistemi.
- Organizasyon firması veya bayi paneli.
- AI, yüz tanıma, yorum/beğeni, slideshow.
- Native iOS/Android uygulaması.
- Google Drive entegrasyonu.
- Gelişmiş kullanıcı yönetimi, sosyal login veya e-posta doğrulama.

3. Teknik Mimari

MVP tek bir Spring Boot backend, tek PostgreSQL, tek React frontend ve Cloudflare R2 object storage kullanır. Backend modular monolith olarak geliştirilir.



Katman	Teknoloji	MVP Notu
Backend	Java 21 + Spring Boot	REST API, modular monolith
Security	Spring Security	Admin koruması
DB	PostgreSQL	Tek instance
ORM	Spring Data JPA	Minimal repository yapısı
Frontend	React + Vite	Mobile-first
Storage	Cloudflare R2	S3-compatible
QR	ZXing	Public URL QR PNG
Containers	Docker Compose	Local + prod
Proxy	Nginx	HTTPS + routing
CI/CD	Jenkins	Basit pipeline
Registry	GHCR veya Docker Hub	Docker image saklama

Repository yapısı

```
wedding-share/
|-- backend/
|   |-- src/
|   |-- pom.xml
|   `-- Dockerfile
|-- frontend/
|   |-- src/
|   |-- package.json
|   `-- Dockerfile
|-- infrastructure/
|   |-- docker-compose.yml
|   `-- nginx/
|-- Jenkinsfile
|-- AGENTS.md
|-- README.md
```

```
|-- .env.example
`-- .gitignore
```

4. Veri Modeli

```
users
- id
- email
- password_hash
- created_at

events
- id
- owner_id
- name
- event_date
- public_token
- cover_image_key
- active
- created_at

media
- id
- event_id
- storage_key
- original_filename
- mime_type
- size_bytes
- created_at
```

İlişkiler: User 1-N Event, Event 1-N Media. Public URL hiçbir zaman database ID'sini dışarı açmaz; tahmin edilmesi zor ve benzersiz public_token kullanılır.

5. Upload Mimarisi

Fotoğraf ve videolar Spring Boot üzerinden taşınmaz. Backend kısa ömürlü presigned upload URL üretir; browser dosyayı doğrudan R2'ye gönderir.

1. Guest -> POST /api/public/events/{token}/uploads/presign
2. Backend -> event aktif mi + MIME + boyut kontrolü
3. Backend -> R2 presigned URL + storage key
4. Browser -> R2'ye direkt PUT
5. Browser -> POST /api/public/events/{token}/media
6. Backend -> media kaydını PostgreSQL'e ekler

Başlangıç limitleri

- Bir seçimde maksimum 30 dosya.
- Fotoğraf maksimum 20 MB.
- Video maksimum 250 MB.
- Sadece izin verilen image/video MIME türleri.
- Presigned URL kısa ömürlü olmalı.
- Her dosya için ayrı upload progress ve retry tutulmalı.

6. Admin İndirme Modeli

Admin galerisinde üç indirme modu zorunludur. Amaç yüzlerce medya içeren etkinliklerde yönetimi pratik tutmaktır.

1 - Tekil indirme

Her medya kartında doğrudan indirme aksiyonu bulunur. Tek dosya için ZIP oluşturulmaz.

2 - Seçilenleri indirme

Galeride checkbox ile istenen medya seçilir. Admin Seçilenleri indir dediğinde backend seçilen media ID'lerini doğrular ve ZIP stream döndürür.

3 - Tümünü indirme

Admin Tümünü indir dediğinde etkinliğe ait tüm medya tek ZIP stream olarak indirilir.

```
GET /api/events/{eventId}/media/{mediaId}/download
```

```
POST /api/events/{eventId}/media/download
{
  "mediaIds": ["uuid-1", "uuid-2"]
}
```

```
GET /api/events/{eventId}/media/download-all
```

MVP'de background ZIP job, queue veya export worker yoktur. Çok büyük etkinliklerde ihtiyaç doğarsa async export post-MVP olarak eklenebilir.

7. API Yüzeyi

```
POST /api/auth/login
```

```
POST /api/events
```

```
GET /api/events
```

```
GET /api/events/{id}
PUT /api/events/{id}

GET /api/events/{id}/media
GET /api/events/{id}/media/{mediaId}/download
POST /api/events/{id}/media/download
GET /api/events/{id}/media/download-all
GET /api/events/{id}/qr

GET /api/public/events/{token}
POST /api/public/events/{token}/uploads/presign
POST /api/public/events/{token}/media
```

Aktif milestone açıkça gerektirmedikçe Codex yeni endpoint veya servis eklememelidir.

8. Mobile-First Frontend

Misafir tarafının ana cihazı telefondur. Tasarım ve test önceliği: 1) iPhone Safari, 2) Android Chrome, 3) Desktop.

[Kapak Fotoğrafı]

Ayşe & Mehmet
12 Eylül 2026

Anılarımızı bizimle paylaş ♥

[Fotoğraf / Video Seç]

IMG_1012.jpg Hazır
IMG_1013.jpg %72
video.mov Hata - Tekrar dene

[YÜKLE]

- Büyük dokunma alanları.
- Tek elle kullanılabilir basit akış.
- Misafir için menü, kayıt veya onboarding yok.
- multiple file input kullanılır.
- Seçili dosya sayısı ve dosya bazlı durum görünür.
- Kötü mobil bağlantıda tek dosya hatası diğerlerini iptal etmemelidir.
- Desktop desteklenir ancak tasarım önceliği değildir.

9. Admin Galerisi

Admin galerisinde medya grid olarak görüntülenir. Her kartta preview, dosya tipi ve tekil indirme aksiyonu bulunur. Üst toolbar seçim durumuna göre değişir.

[Tümünü Seç] [Seçilenleri İndir] [Tümünü İndir]

[x] IMG_001.jpg
[] IMG_002.jpg
[x] VID_003.mov
[] IMG_004.jpg

Seçilenleri indir butonu yalnızca en az bir medya seçildiğinde aktif olmalıdır.

10. Minimum Güvenlik

- Admin endpointleri authentication gerektirir.
- Public endpointler yalnızca public_token ile çalışır.
- public_token tahmin edilemez ve benzersizdir.
- Upload öncesi event active, MIME ve size kontrol edilir.
- R2 secret credential hiçbir zaman frontend'e gönderilmez.
- Secrets environment variable olarak tutulur.
- Production HTTPS zorunludur.

- Public upload için basit rate limiting production öncesi eklenebilir; distributed rate limiter kurulmaz.

11. CI/CD Hedefi

CI/CD önemlidir ancak enterprise seviyesine çıkarılmaz. Amaç geliştiricinin main branch'e push etmesinden sonra test/build/image/deploy işlemlerinin otomatik yapılmasıdır.

```
Developer
|
| git push origin main
v
GitHub
|
| webhook
v
Jenkins
|
+--> Backend test
+--> Frontend build/test
+--> Docker images build
+--> Registry push
|
v
Production VPS
|
+--> docker compose pull
`--> docker compose up -d
```

Jenkins pipeline kapsamı

- Repository checkout.
- Backend test ve package/build.
- Frontend dependency install ve production build.
- Backend ve frontend Docker image build.
- Image'ların commit SHA veya build number ile taglenmesi.
- GHCR veya Docker Hub'a push.
- Production VPS'e güvenli SSH bağlantısı.
- docker compose pull.
- docker compose up -d.
- Deploy başarısızsa pipeline fail olmalı.

CI/CD kapsamında olmayanlar

- Blue/green deployment.
- Canary release.
- Kubernetes.
- ArgoCD.
- Terraform.
- Multi-node Jenkins veya distributed agents.
- Karmaşık manual approval zincirleri.
- Dev/stage/prod gibi çoklu environment yönetimi.

Image tagging

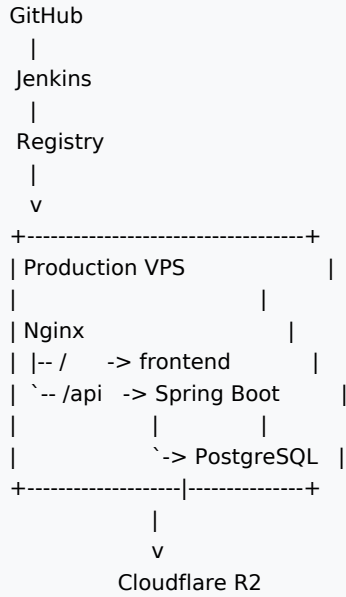
```
wedding-share-backend:<git-sha>  
wedding-share-frontend:<git-sha>
```

Opsiyonel:

```
wedding-share-backend:latest  
wedding-share-frontend:latest
```

Deploy için mümkünse immutable SHA tag kullanılması tercih edilir; latest yalnızca kolaylık etiketi olarak tutulabilir.

12. Production Topolojisi



Jenkins ilk aşamada aynı VPS'te ayrı container olarak çalıştırılabilir; ancak uygulama container'larından mantıksal olarak ayrılır. Daha sonra ihtiyaç olursa ayrı CI sunucusuna taşınabilir.

13. Local -> Production Akışı

```
LOCAL DEVELOPMENT
Spring Boot + React + PostgreSQL
|
v
Docker Compose local verification
|
v
Real device testing
iPhone Safari / Android Chrome
|
v
git push main
|
v
Jenkins pipeline
|
v
Docker Registry
|
v
Production VPS + Nginx + HTTPS + Domain
```

Production'a geçerken yeni bir mimari oluşturulmaz. Localde doğrulanan container yaklaşımı production'a taşınır.

14. Environment Variables

```
DB_URL=
DB_USERNAME=
DB_PASSWORD=

R2_ENDPOINT=
R2_ACCESS_KEY_ID=
R2_SECRET_ACCESS_KEY=
R2_BUCKET=

APP_PUBLIC_BASE_URL=

ADMIN_EMAIL=
ADMIN_PASSWORD=

REGISTRY_USERNAME=
REGISTRY_TOKEN=

DEPLOY_HOST=
DEPLOY_USER=
```

Gerçek secret değerleri repository'ye commit edilmez. Jenkins credentials store veya production environment kullanılır.

15. Codex Yol Haritası - M0 to M19

ID	Milestone	Beklenen çıktı
M0	Repository Bootstrap	Repo yapısı, AGENTS, README, env örneği.
M1	Spring Boot Bootstrap	Java 21 minimal backend ve build.

ID	Milestone	Beklenen çıktı
M2	PostgreSQL + Entities	users/events/media persistence.
M3	Admin Authentication	Minimal admin login.
M4	Event CRUD	Create/list/get/update event.
M5	Public Event	public_token ile guest event verisi.
M6	Cloudflare R2	S3-compatible storage config/client.
M7	Presigned Upload	Upload izin akışı + validasyon.
M8	Media Registration	R2 upload sonrası DB media kaydı.
M9	React Bootstrap	Vite, routing, API client.
M10	Mobile Guest Upload	Mobile UI, multi-file, progress, retry.
M11	Admin Gallery	Media grid, selection state.
M12	Media Download	Tekil + seçilen ZIP + tümü ZIP.
M13	QR Generation	Public event URL için QR.
M14	Local Docker Compose	Frontend/backend/db local stack.
M15	Mobile Testing	Safari/Chrome gerçek cihaz fixleri.
M16	Production Docker	Nginx, HTTPS-ready production compose.
M17	Jenkins CI	Test/build/docker image/registry.
M18	Jenkins CD	main push -> VPS auto deploy.
M19	Production Launch	Domain, HTTPS, smoke test, first event.

16. Codex Çalışma Kuralları

Bu belge repository için ana mimari kaynaktır. Codex her milestone'da mevcut repository'yi incelemeli, yalnızca aktif milestone kapsamını uygulamalı ve sistemi gereksiz yere büyütmemelidir.

CORE RULES

- This is a commercial MVP.
- Optimize for fast delivery and simple maintainable code.
- Backend: Java 21 + Spring Boot.
- Frontend: React + Vite.
- Database: PostgreSQL.
- Storage: Cloudflare R2 / S3-compatible.
- Architecture: modular monolith.
- Guest UX: mobile-first.
- Priority browsers: iPhone Safari, Android Chrome.
- Deployment: Docker Compose on one VPS.
- CI/CD: GitHub -> Jenkins -> Registry -> VPS.

DO NOT ADD

- Microservices
- Kubernetes
- Redis
- Kafka
- RabbitMQ
- Terraform
- ArgoCD
- Blue/green or canary deployment
- Features outside active milestone
- Enterprise abstractions without a concrete need

BEFORE IMPLEMENTATION

1. Inspect repository.
2. Identify current milestone.
3. Read this architecture document.
4. Explain understanding/plan in max 8 bullets.
5. List files expected to change.
6. If interactive, wait for approval before modifications.

AFTER IMPLEMENTATION

1. Run relevant tests.
2. Run backend/frontend build.
3. Report exactly what changed.
4. Report test/build result.
5. Report blockers honestly.
6. Do not silently expand scope.

Codex karar kuralı

Bir teknoloji veya mimari karar bu belgede açıkça verilmişse Codex alternatif araştırıp yapıyı değiştirmemelidir. Ancak gerçek bir teknik blocker varsa bunu implementasyondan önce açıkça raporlamalıdır.

17. Definition of Done - MVP

- Admin login çalışıyor.
- Etkinlik oluşturuluyor ve public URL üretiliyor.
- QR PNG indirilebiliyor ve telefondan okutulabiliyor.

- Guest sayfası iPhone Safari ve Android Chrome'da kullanılabilir.
- Misafir birden fazla fotoğraf/video seçebiliyor.
- Dosyalar doğrudan R2'ye upload ediliyor.
- Media kayıtları PostgreSQL'e yazılıyor.
- Admin galeride medyaları görüntüleyebiliyor.
- Admin tek dosya indirebiliyor.
- Admin seçtiği dosyaları ZIP olarak indirebiliyor.
- Admin etkinliğin tüm medyasını ZIP olarak indirebiliyor.
- Local stack Docker Compose ile ayağa kalkıyor.
- Backend ve frontend test/build Jenkins'te çalışıyor.
- Docker image registry'ye push ediliyor.
- main push production VPS'te otomatik deploy tetikliyor.
- Domain + HTTPS ile gerçek cihazdan uçtan uca smoke test geçiyor.

18. Son Direktif

Codex bu belgeyi proje sözleşmesi gibi kullanmalıdır. Amaç M0'dan M19'a kadar sırayla ilerleyerek her milestone'u küçük, build edilebilir ve test edilebilir halde bitirmektir.

Tüm sistemi tek seferde üretmeye çalışma. Aktif milestone'un dışına çıkma. Gereksiz dokümantasyon veya enterprise mimari üretme.

Nihai hedef: QR okutulduğunda birkaç saniye içinde fotoğraf/video yüklenebilen, adminin medyayı rahatça yönetip üç farklı şekilde indirebildiği ve GitHub push sonrası otomatik production deploy alan ilk ticari MVP.